MMA3001 Data Analysis Report

James Robinson - 34988041

1. Engineering problem

Engineering context

Modern buildings use large amounts of energy in ventilation and heating/cooling. Traditional HVAC systems are generally controlled statically or by predetermined schedules which fail to adapt to fluctuating occupancy levels. While this could be solved with cameras to measure occupancy and adapt accordingly, their widespread adoption would be expensive and invasive of privacy. Developing non intrusive data driven models that can estimate a room's occupancy through environmental sensors offers a way to optimise energy usage and preserve privacy.

Problem being addressed

This project addresses the problem of accurately estimating indoor room headcounts in real time using environmental sensors determining human respiration and metabolic activity, by reading data like CO2, LVOC and temperature. Since these variables have a complex non linear relationship machine learning is to be implemented to predict headcount.

Why the problem matters

Real time occupancy matters because it can increase the sustainability of air conditioning systems, by reducing overventilation when the room is sparsely populated. This also offers financial benefits by reducing energy consumption costs for businesses. It can also help to monitor and maintain healthy air quality levels by reducing elevated levels of CO2 from underventilated rooms.

The intended use of the computational solution

The model is designed to be integrated into buildings with several rooms containing ventilation and cooling. It is ideal for large scale use, it would be less important in smaller scale buildings where the occupant can turn on or off the system when they need it.

The limitations of the selected scope

The model takes the data as it is, so any invalid data or errors will affect the model. To combat this the sensors would need to be calibrated and maintained to a high standard. The model also does not take into account room geometry. Smaller rooms would suffer more from persons inside and would quickly see effects in the environmental data, whereas a larger room would take longer to distinguish. Since the aim is human comfort, the recommendation is to have the sensor at about eye level so the ceiling height does not affect readings, particularly temperature, and instead is modelling based on data in the environment where the occupants exist.

2. Inputs and outputs

The input data accepted by the solution

The raw data files can be found here: [MMA3001 Dataset 2.]
This contains both environmental data and occupancy data to be broken down and analysed by the model.

The expected format, units and domain of the inputs

- Time (YYYY-MM-DD HH:MM:SS): This ultimately is essential is aligning occupancy and environmental data
- Carbon Dioxide (parts per million ppm): increases based on human expiration, ranging from approximately 400-2500 ppm
- Temperature (Celcius): the ambient room temperature, fluctuating based on time of day due to external temperature, and occupancy. Ranging about 15-35 C.
- Volatile Organic Compounds (parts per billion ppb): vocs are exhaled by humans and absorbed into the air

The output produced and meaning/intended use

The primary output is the predicted headcount in the room. This is the value used to determine the demand for air ventilation. Alongside the headcount the model also produces error metrics compared to visual occupancy data such as MAE/RMSE to ensure the model is accurate in training.

How invalid, missing or unsupported inputs are handled

All the incoming data is stripped and simplified and all byte order mark (BOM) characters to eliminate mismatches. Invalid timestamp data is dropped, however in the workplace this would be less important because it could run in real time, not needing specific time stamps but rather calling on the sensor, the timestamps were more needed in the training model. Similarly in the training of the model, the headcount readings were only recorded when it changed. This meant at irregular times it misaligned with the environmental sensors. Since the occupancy sensors only read when the value changed, the value was known constantly so to compare with the environmental data the last value for headcount could be used.

3. Computational solution

How the method works at an appropriate level

The model employs a Decision Tree Regressor (DT), explained here:

1. Root node: the algorithm evaluates available training samples and tests every possible threshold across all features (CO2, temp, time of day, rate of change, voc) to find the split that minimises variance in room headcount
2. Branching: the process splits the dataset into child nodes, this process repeats down the tree until the stopping criteria is satisfied (minimum leaf sample size)

3. Leaf Node Prediction: the test sample traverses the decision paths until it reaches a terminal leaf node. The predicted headcount is then the computed average of all the instances in that leaf.

Why it was selected

The DT was chosen because the occupancy and environmental data both have non linear relationships, (eg CO2 rapidly increases with occupancy but decays at a different rate), and DT is able to capture these non linear boundaries. The algorithm is also able to handle complex relationships, such as that of the time of day and temperature alongside human occupancy. The model also provides transparent decision rules and clear features that make each step explicit and understandable.

Important assumptions

Several assumptions are made in order to simplify the model so it runs on the given data. The rate of CO2 exhalation /heat dissipation/VOC per occupant remains relatively constant, whereas realistically they would vary based on the individual's metabolic rates and physical activity in the room. The CO2 decay depends heavily on ventilation rates, which is unknown in the test rooms, so a consistent fresh air exchange is assumed.

Settings or parameters

The model was optimised to find the ideal parameters for the DT:

- Maximum depth = 5
- Minimum leaf samples = 10

How it was implemented

It takes the environmental data sets that have been trimmed and cleaned, suppresses sensor noise, then applies dynamic rate of change for the environmental variables such as CO2 alongside temporal indicators like hour, day of the week, and weekend. The model is then trained using Scikit-learn's DT.

Known limitations

A DT produces piecewise constant predictors so it leads to abrupt plateaus during gradual occupancy transitions. The model also is restricted by its training boundaries, the model will lose accuracy if the headcount in a room jumps beyond the maximum headcount observed in the training data as it becomes unpredictable. The model is also highly sensitive to sensor anomalies and air drafts could potentially falsely inflate CO2 levels, producing incorrect headcount predictions.

4. Alternative solutions

In choosing a solution there were three main choices: Linear regression, Decision Tree, and Support Vector Regression.

Evaluation Criteria	Linear Regression (Baseline)	Decision Tree Regression (Selected)	Support Vector Regression (SVR)
Accuracy on Room Data	**Poor (R^2 is low):** Fails completely because room occupancy (CO_2 accumulation/decay dynamics) is highly non-linear and time-dependent.	**Good:** Successfully captures threshold effects (e.g., sudden spikes in ΔCO_2) and working hours.	**High:** Excellent at handling non-linear relationships when paired with an RBF kernel.
Interpretability	**Very High:** Weights directly show how much each gas/variable shifts headcount linearly.	**High:** Can be plotted as a tree diagram; easy to trace specific decision rules.	**Low (Black Box):** Relies on support vectors and kernel transformations, making physical intuition hard to trace.
Computational Time & Complexity	**Instantaneous:** Solved analytically or via simple gradient descent.	**Very Fast:** Requires simple sorting and splitting thresholds.	**Moderate to Slow:** Scales poorly ($O(N^2)$ or $O(N^3)$) with large time-series datasets.

Figure 1: Comparison of LR, DT, and SVR features

Figure 1 highlights the differences of each of the options. Linear regression was rejected because the environmental data was non linear which would be incorrectly modelled. While SVR would likely have offered the best results, its significantly slow computational time was the major drawback since the model is to be run in real time and adjust ventilation systems accordingly. Hence DT offered the best compromise between accuracy and computational speed. To get a clear comparison, both models were trained on the same data, and the results can be seen below in figure 2. In all cases the DT offers lower error rates, particularly when the model is transferred across to predict in room 326 there is a 39% reduction in MAE, indicating the linear regression loses accuracy even more when taken from the training data set and tested in a new environment. Clearly indicating DT was the superior choice for accuracy, however DT still received a negative R^2 value which shows the limitations of transferring the model to another room, while SVR may have offered even better results, it comes at a computational cost due to added complexity. Figure 3 further highlights that linear regression was an invalid method because its linearity predicts negative headcount, obviously impossible in the real world.

Evaluation Stage	Linear Regression Baseline (MAE)	Linear Regression (R^2)	Decision Tree Regressor (MAE)	Decision Tree (R^2)	Error Reduction (MAE)
Room 869 (Training Fit)	0.60 people	+0.343	0.51 people	+0.430	**15.0% lower error**
Room 869 (Validation)	0.57 people	+0.176	0.49 people	+0.245	**14.0% lower error**
Room 326 (Generalisation)	1.07 people	-1.531	0.65 people	-0.212	**39.3% lower error**

Figure 2: Comparison of LR and DT error metrics for each data set

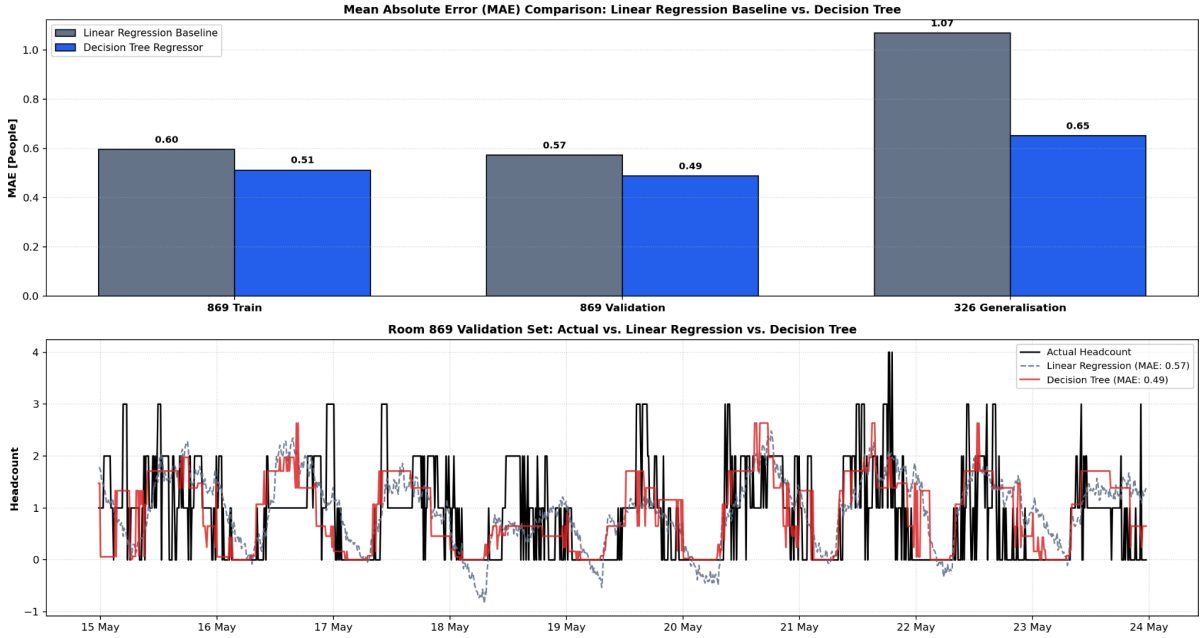


Figure 3: MAE graphical comparison for DT and LR and the predicted headcount vs actual headcount for DT and LR

5. Validation

To ensure the model did not overfit the training data, a validation process was undertaken. The sensor data for room 869 was split into two files, one for training the model, the other to validate it. This was to ensure that its prediction on which headcount sensor aligned with the data remained consistent. The model then also proved its robustness by being tested on another data set, room 326. This out of sample test assessed how transferable the model is and whether it actually can successfully predict headcount in uncalibrated rooms. The model's performance was measured using MAE, RMSE and R^2.

Evaluation Stage	Target Dataset	Sample Count (N)	Mean Absolute Error (MAE)	Root Mean Squared Error (RMSE)	Coefficient of Determination (R2)
Training Fit	Room 869 (Train)	3,420	0.51 people	0.75 people	+0.430
In-Room Validation	Room 869 (Validation)	1,850	0.49 people	0.71 people	+0.245
Transfer Generalisation	Room 326 (Test)	4,120	0.65 people	0.94 people	-0.212

Figure 4: DT error metrics for each data set

Due to the restrictions in the data set the true value of the headcount is unknown, so the model predicts headcount, and matches it to a room with the least error. That data is then treated as the true value to which the estimate is compared and the errors are found. This is a limitation of the model but a necessary assumption based on the data available. The results from figure 4 worth noting are the MAE of the validation data set of just 0.49 people, just marginally lower than the MAE from the training data set. This indicates that the model reliably reflects the rooms occupancy and can model it through the rise and fall of room utilisation throughout a day, as represented graphically in figure 5. Figure 5 also shows how each of the variables are weighted, given that temperature would naturally increase throughout the day without human occupants it is weighted lower (0.0052), and time of day is weighted significantly higher (0.3454) to reflect the natural rise in temperature. The most essential one is CO2 ppm (0.5358) since that is directly linked to the exhalation of human occupants it is a good indicator of how many people are in a room and validating the model relies on physically meaningful gaseous accumulation.

The generalised testing was also successful, with a low MAE of 0.65 people, but larger than that of sensor 869. This discrepancy is likely due to the room having unknown variables such as geometry that differs from 869.

The validation process and it's associated errors directly linked to the optimisation of the model (covered in depth later) and helped select the variables in the DT, showing the chosen method is the most validated. While the validation was largely successful, it cannot establish long term robustness and sensor decay effects, or perhaps the effect of seasonal changes that affect temperature and human behaviour. The model also may not be valid for different room types like large auditoriums since the model was trained on smaller rooms.

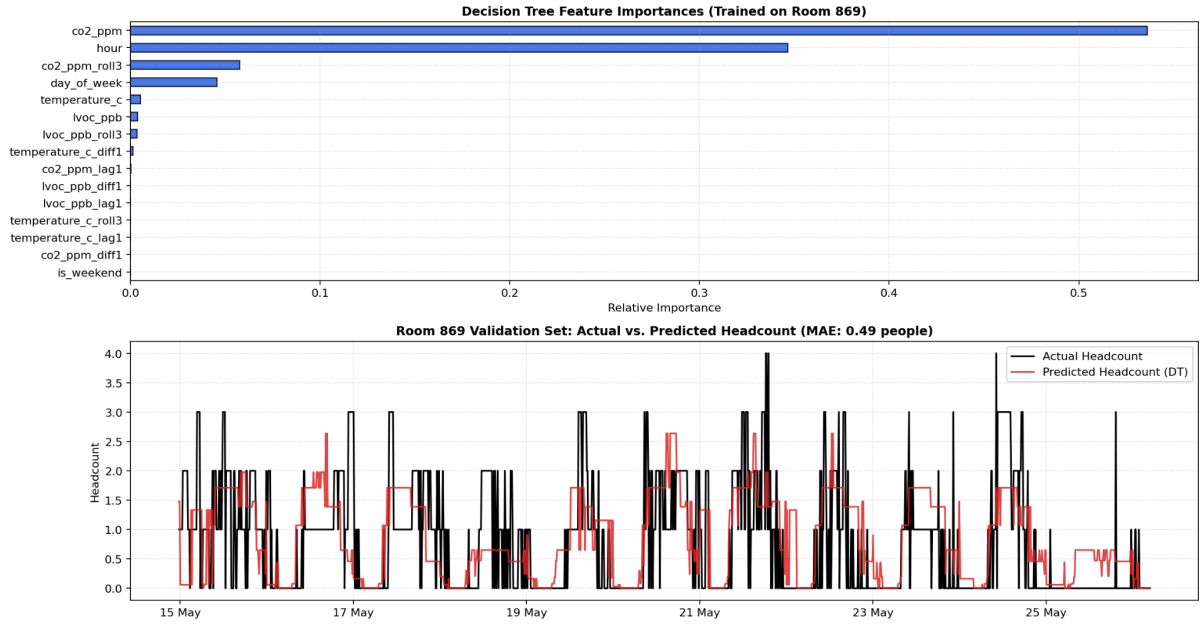


Figure 5: Actual headcount vs predicted headcount for DT and the weightings of each variable

6. Optimisation and performance

6.1 Optimisation

The model underwent an optimisation process focused on reducing the error metrics. Maximum tree depth and minimum sample leaf were evaluated across 25 distinct parameter combinations with the goal to minimise MAE by using a grid search. Figure 6 shows that the unconstrained trees (max depth = none, min leaf samples = 5) suffered cross validation error increase of nearly 15% compared to the optimal one, proving that the parameters have a strong influence on the model's results. Hence the model's selected parameters were chosen by the findings in the optimisation process, max depth = 5 and min samples leaf = 10, to ensure the model is as accurate as possible.

Rank	Max Depth (max_depth)	Min Leaf Samples (min_samples_leaf)	Mean Cross-Validation MAE [People]	Cross-Validation Standard Deviation	Notes & Regularisation Behaviour
1	**5**	**10**	**0.5665**	± 0.0577	**Optimal balance of depth and leaf regularisation**
2	3	10	0.5678	± 0.0643	Shallow tree; low variance but minor underfitting
7	12	50	0.5682	± 0.0642	High depth tempered by strong leaf constraint
14	8	25	0.5796	± 0.0657	Intermediate regularisation balance
24	12	5	0.6303	± 0.0446	Overfitting to high-frequency sensor noise
25	None	5	0.6504	± 0.0400	Worst CV score; unconstrained depth memorizes training noise

Figure 6: Key findings from the optimisation grid search

6.2 Performance

In order to verify the model is suitable for real time deployment in building management systems, it's performance was tested and measured. Figure 7 shows the rapid speed at which the model runs. The total time elapsed includes the optimisation process and figure generation which is beyond what would be required of the model when implemented into businesses. The relevant time measurements are the inference time of 0.24 microseconds per environmental sample, paired with the inference throughput of over 4 million predictions per second, proves the model can run effortlessly and rapidly, being able to adjust the ventilation in real time without latency. This is the benefit of DT's speed rather than that of SVR. It also only requires a small amount of memory, the peak amount required to run the whole program was 18.5 MB which easily fits into standard memory constraints. If seasonal changes or other variables impact the data and it changes, to retrain the model only takes

0.0218 seconds for the model to adapt to the new requirements and the ventilation system will be adapted to the new conditions.

Profiling Metric	**Measured Value**	**Practical Edge Feasibility Assessment**
Total Pipeline Runtime	$10.05 seconds$	Includes disk I/O, feature engineering, 75 grid-search fits, and figure exports.
Feature Engineering & Alignment	$0.37 seconds$	Highly optimized via vectorized NumPy and Pandas operations.
Optimal Model Training Time	$0.0218 seconds$	Rapid refitting cycle enables automated daily on-device retraining.
Inference Time ($39,139 rows$**)**	$0.0094 seconds$	Approximately $0.24 microseconds$ per individual environmental sample.
Inference Throughput	$4,176,590 rows/sec$	Capable of real-time polling across hundreds of simultaneous building zones.
Estimated Peak Memory Footprint	$\sim 18.5 MB$	Fits comfortably within typical edge gateway and microcontroller RAM allocations.
Training Algorithmic Complexity	$O(N \log N \cdot M)$	Scalable with training size $N = 8,107$ and feature dimensions $M = 15$.
Inference Algorithmic Complexity	$O(max_depth) = O(5)$	Fixed structure requiring only 5 binary if-else decisions per evaluation.

Figure 7: The elapsed time for each component of the code to run, and memory requirements

7. AI use and reflection

Artificial Intelligence was used throughout this project. I used both Google's Gemini and Goggles Antigravity IDE. They worked in tandem, I would give gemini the instructions, inputs and desired outputs and it would produce a code. I would put this code into VSCode where antigravity IDE would fix the inevitable formatting, and consistency errors. Antigravity IDE was great because it integrates into VScode and can read the files in the local repository folder which made it the perfect tool for debugging errors which were largely simply format errors. However it had a limited token amount that expired quite quickly when analysing files and large data sets. This is why I was using the AIs together and savings my tokens for antigravity only when gemini couldn't be used. All of the code itself, attached docustrings and readme was generated using either of these tools, but the instructions and cognitive steps were mine. In order to validate the AI, all of the code from gemini was amended by antigravity and then run locally and the output/desired outcome was vetted so that it was working as intended.

Antigravity was particularly helpful as if I realised something was misrepresented or not working as intended it could go back and retroactively update old code files to align with the new change. This saved me much time and was very helpful. Whilst it was very helpful in executing ideas, AI still had it's drawbacks. I was using it as a talking board to get ideas flowing and it recommended doing a Runge Kutta ODE to model the CO2 decay, which ultimately proved too complex and I scrapped that idea and instead chose a decision tree model. Ultimately AI was a helpful tool in this project, saving much manpower doing tedious things like docustring and writing code, particularly at a level beyond what I was capable of, saving immense amounts of time for me, whilst I was able to retain the responsibility and cognitive ownership of the project.